

Assignment 3

Kumar Kanishk Singh

2024-03-11

Question1

A

```
Y<-c(12,90,80,5,63,15,67,22,56,33)
X<-c(50,150,63,10,63,8,56,19,63,19)
N<-X+Y
n<-length(Y)

#a

alpha<-Y+1/2
beta<-N-Y+1/2

theta.Y<-rbeta(n,alpha,beta)

# Set the confidence level
cl <- 0.95

lower_quantile <- qbeta((1 - cl) / 2, alpha, beta)
upper_quantile <- qbeta(1 - (1 - cl) / 2, alpha, beta)
CI<-cbind(lower_quantile,upper_quantile)
print(CI)
```

```
##      lower_quantile upper_quantile
## [1,]      0.1103935      0.3045307
## [2,]      0.3155448      0.4374498
## [3,]      0.4775597      0.6389423
## [4,]      0.1402526      0.5841638
## [5,]      0.4135270      0.5864730
## [6,]      0.4488766      0.8197773
## [7,]      0.4565230      0.6308309
## [8,]      0.3858303      0.6823739
## [9,]      0.3825161      0.5600718
## [10,]     0.4993247      0.7553816
```

B

```
p<-Y/N

mean.p<-mean(p)
var.p<-var(p)
```

```

M<-mean.p
V<-var.p

a<-(((M^2)*(1-M))-M*V)/V
b<-(M*(1-M)^2-(1-M)*V)/V
print("a:")

```

```
## [1] "a:"
```

```
print(a)
```

```
## [1] 5.433856
```

```
print("b:")
```

```
## [1] "b:"
```

```
print(b)
```

```
## [1] 5.886676
```

C

```

alpha2<-a+Y
beta2<-N-Y+b

# Set the confidence level
cl <- 0.95

lower_quantile2<- qbeta((1 - cl) / 2, alpha2, beta2)
upper_quantile2 <- qbeta(1 - (1 - cl) / 2, alpha2, beta2)
CI2<-cbind(lower_quantile2,upper_quantile2)
print(CI2)

```

```

##      lower_quantile2 upper_quantile2
## [1,]      0.1482134      0.3408754
## [2,]      0.3208095      0.4404631
## [3,]      0.4748854      0.6310240
## [4,]      0.2221416      0.5856048
## [5,]      0.4151793      0.5815687
## [6,]      0.4293641      0.7508658
## [7,]      0.4548591      0.6225556
## [8,]      0.3899776      0.6569425
## [9,]      0.3865857      0.5570576
## [10,]     0.4848498      0.7226928

```

Question 3

C

```

sigmaSq.update <- function(n,Y, a, b)
{
  sigmaSq.inv<-rgamma(n,a+1/2,b+(Y^2)/2)
  sigmaSq<-sigmaSq.inv
  return(sigmaSq)
}

```

```

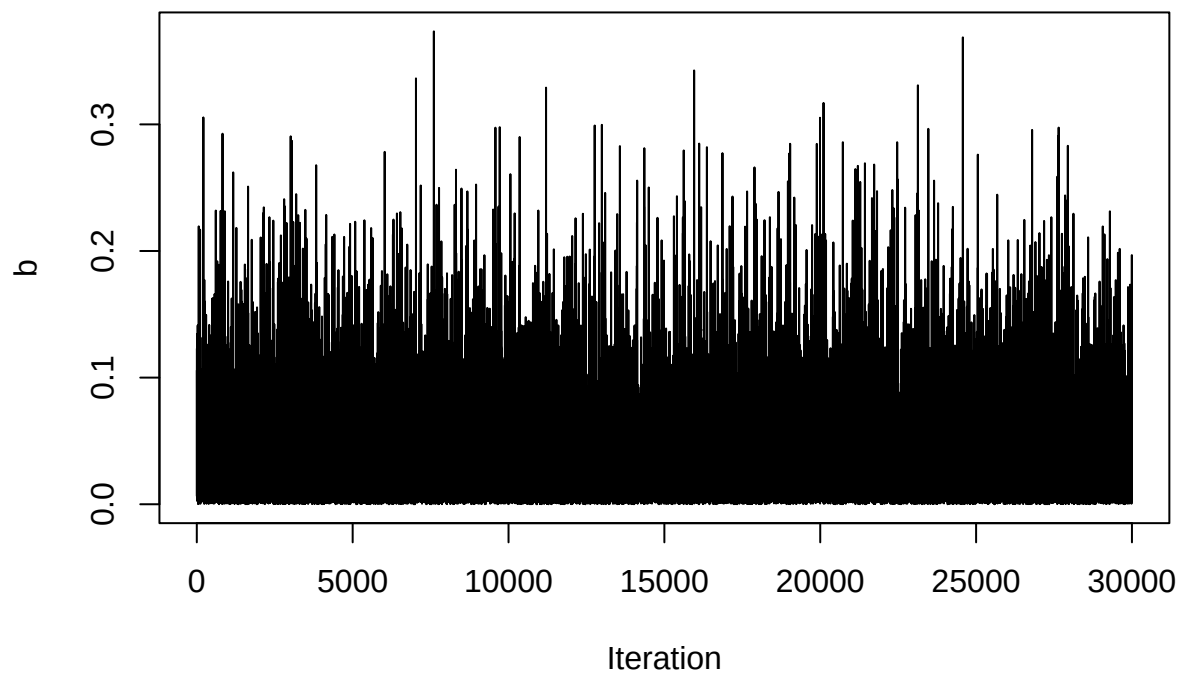
b.update <- function(sigmaSq)
{
  b<-rgamma(1,1,b+sum(1/sigmaSq))
  return(b)
}

MCMC <- function(n,Y, b.init, sigmaSq.init, a, iters){
  # chain initiation
  b<-b.init
  sigmaSq <- sigmaSq.init
  # define chains
  b.chain <- rep(NA, iters)
  sigmaSq.chain <- matrix(NA, iters,n)
  # start MCMC
  for(i in 1:iters){
    b <- b.update(sigmaSq)
    sigmaSq <- sigmaSq.update(n,Y, a, b)
    b.chain[i] <- b
    sigmaSq.chain[i,] <- sigmaSq
  }
  # return chains
  out <- list(b.chain = b.chain,
             sigmaSq.chain = sigmaSq.chain)
  return(out)}

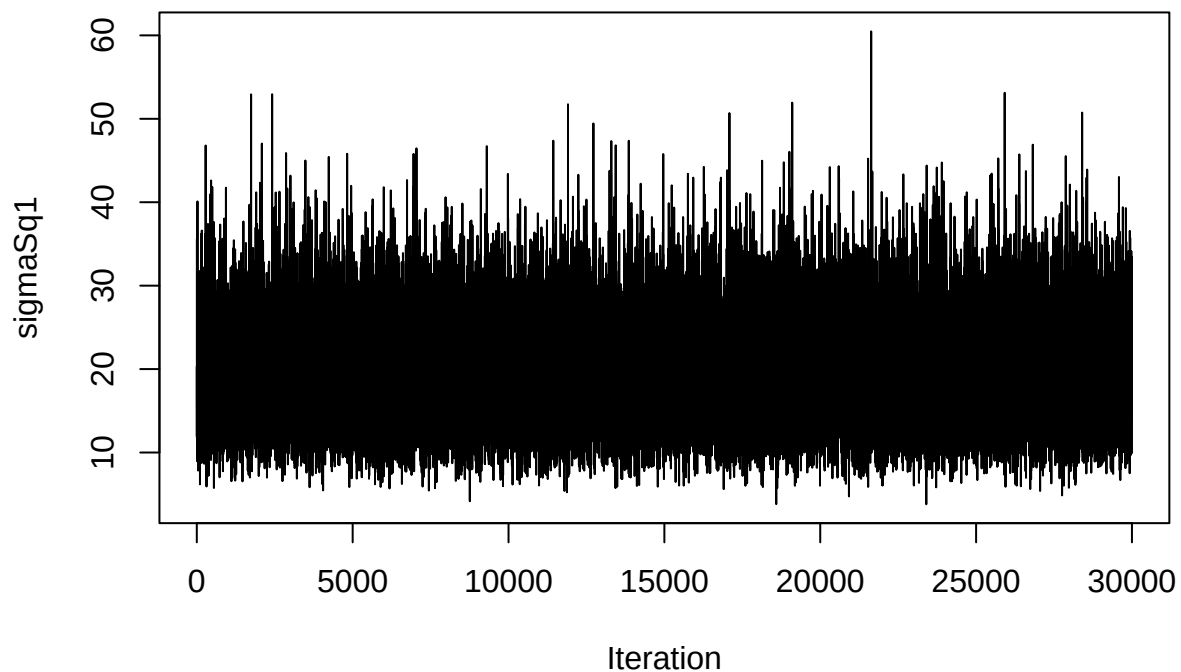
N<-10
Y<-seq(1:10)
a<-10

MCMC.out <- MCMC(n=N,Y = Y,
                 b.init = 0.1,
                 sigmaSq.init = var(Y),
                 a = a,
                 iters = 30000)
plot(MCMC.out$b.chain, xlab = "Iteration", ylab = "b", type = "l")

```

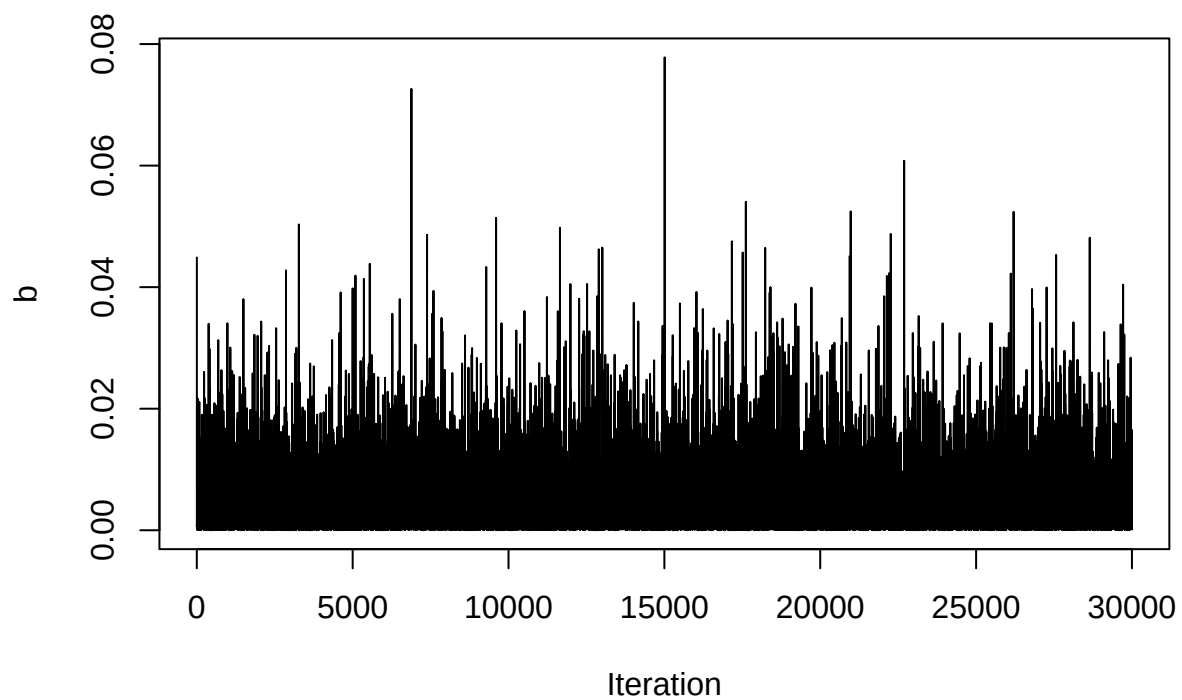


```
plot(MCMC.out$sigmaSq.chain[,1], xlab = "Iteration", ylab = "sigmaSq1", type = "l")
```

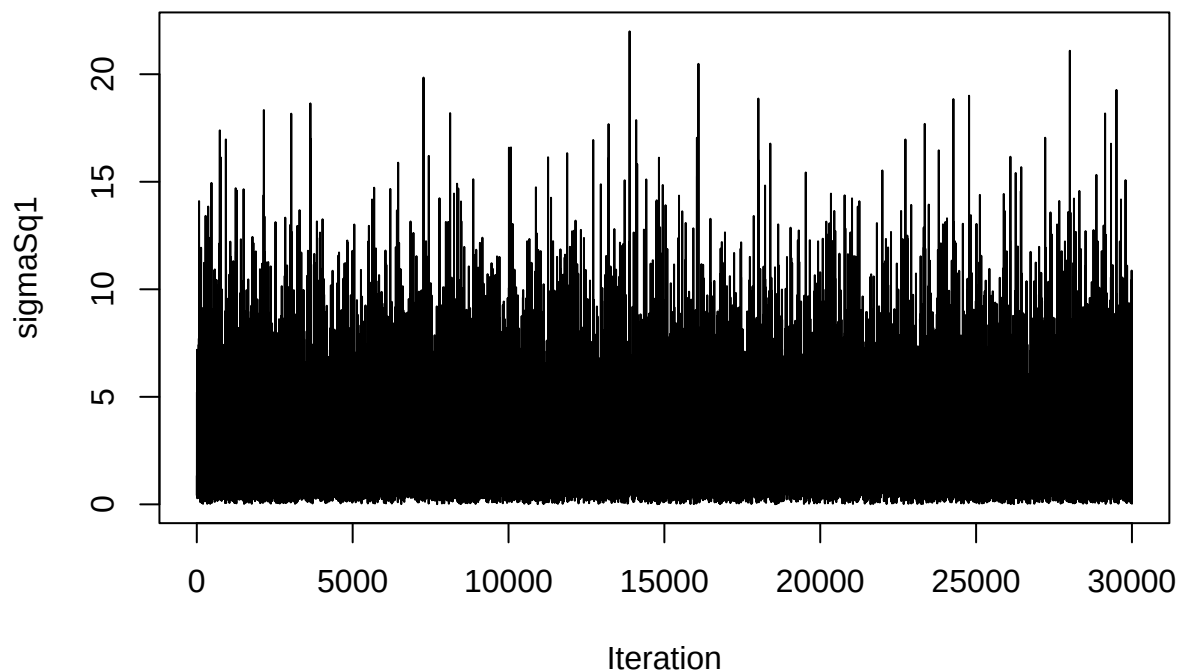


```
#### D
N<-10
Y<-seq(1:10)
a<-1

MCMC.out <- MCMC(n=N,Y = Y,
                 b.init = 0.1,
                 sigmaSq.init = var(Y),
                 a = a,
                 iters = 30000)
plot(MCMC.out$b.chain, xlab = "Iteration", ylab = "b", type = "l")
```



```
plot(MCMC.out$sigmaSq.chain[,1], xlab = "Iteration", ylab = "sigmaSq1", type = "l")
```



```
### E
```

```
library(rjags)
```

```
## Warning: package 'rjags' was built under R version 4.2.3
```

```
## Loading required package: coda
```

```
## Warning: package 'coda' was built under R version 4.2.3
```

```
## Linked to JAGS 4.3.1
```

```
## Loaded modules: basemod,bugs
```

```
n<-10
```

```
Y<-seq(1:10)
```

```
a<-10
```

```
data <- list(Y = Y, n = n)
```

```
model_string <- textConnection("model{
```

```
  # Likelihood
  for(i in 1:n){
    Y[i] ~ dnorm(0,sigma_sq[i])
  }
```

```
  # Priors
  for(i in 1:n){
    sigma_sq[i] ~ dgamma(10,b)
```

```

    }
    b ~ dgamma(1,1)

  }")

model <- jags.model(model_string, data = data,
                    n.chains = 2, quiet = TRUE)

update(model, 10000, progress.bar = "none")

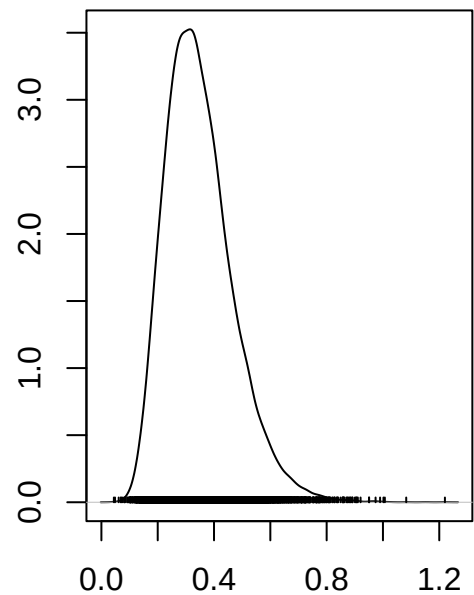
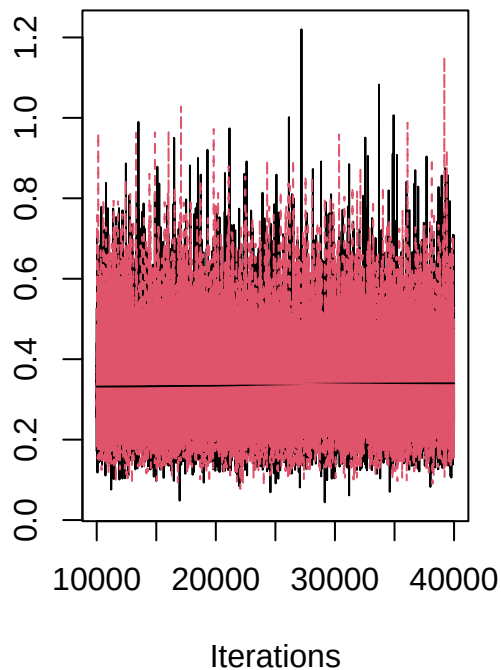
samples <- coda.samples(model, variable.names = "sigma_sq",
                        n.iter = 30000, thin = 2, progress.bar = "none")

summary(samples)

##
## Iterations = 10002:40000
## Thinning interval = 2
## Number of chains = 2
## Sample size per chain = 15000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean      SD Naive SE Time-series SE
## sigma_sq[1] 0.3488 0.12010 0.0006934      0.0007834
## sigma_sq[2] 0.3324 0.11310 0.0006530      0.0007236
## sigma_sq[3] 0.3074 0.10227 0.0005905      0.0006391
## sigma_sq[4] 0.2780 0.09199 0.0005311      0.0005686
## sigma_sq[5] 0.2486 0.08157 0.0004710      0.0005041
## sigma_sq[6] 0.2188 0.07051 0.0004071      0.0004258
## sigma_sq[7] 0.1926 0.06100 0.0003522      0.0003604
## sigma_sq[8] 0.1691 0.05331 0.0003078      0.0003266
## sigma_sq[9] 0.1494 0.04700 0.0002713      0.0002772
## sigma_sq[10] 0.1316 0.04161 0.0002402      0.0002435
##
## 2. Quantiles for each variable:
##
##           2.5%   25%   50%   75%  97.5%
## sigma_sq[1] 0.16133 0.2624 0.3333 0.4171 0.6264
## sigma_sq[2] 0.15454 0.2511 0.3189 0.3972 0.5925
## sigma_sq[3] 0.14461 0.2342 0.2948 0.3672 0.5405
## sigma_sq[4] 0.12987 0.2121 0.2669 0.3327 0.4872
## sigma_sq[5] 0.11714 0.1903 0.2389 0.2956 0.4347
## sigma_sq[6] 0.10603 0.1679 0.2102 0.2606 0.3796
## sigma_sq[7] 0.09255 0.1485 0.1857 0.2295 0.3290
## sigma_sq[8] 0.08235 0.1309 0.1631 0.2014 0.2892
## sigma_sq[9] 0.07151 0.1159 0.1443 0.1777 0.2540
## sigma_sq[10] 0.06352 0.1017 0.1270 0.1569 0.2252

plot(samples[,1])

```

N = 15000 Bandwidth = 0.01557

Question 4

D

```
theta.update <- function(n,Y,Q,N,m)
{
  theta<-rbeta(n,Y+exp(m)*Q,N-Y+exp(m)*(1-Q))
  return(theta)
}

m.update <- function()
{
  m<-rnorm(1,0,10)
  return(m)
}

MCMC <- function(n,Y,Q,N, m.init, theta.init, iters){
  # chain initiation
  m<-m.init
  theta <- theta.init
  # define chains
  m.chain <- rep(NA, iters)
  theta.chain <- matrix(NA, iters,n)
  # start MCMC
  for(i in 1:iters){
    m <- m.update()
```

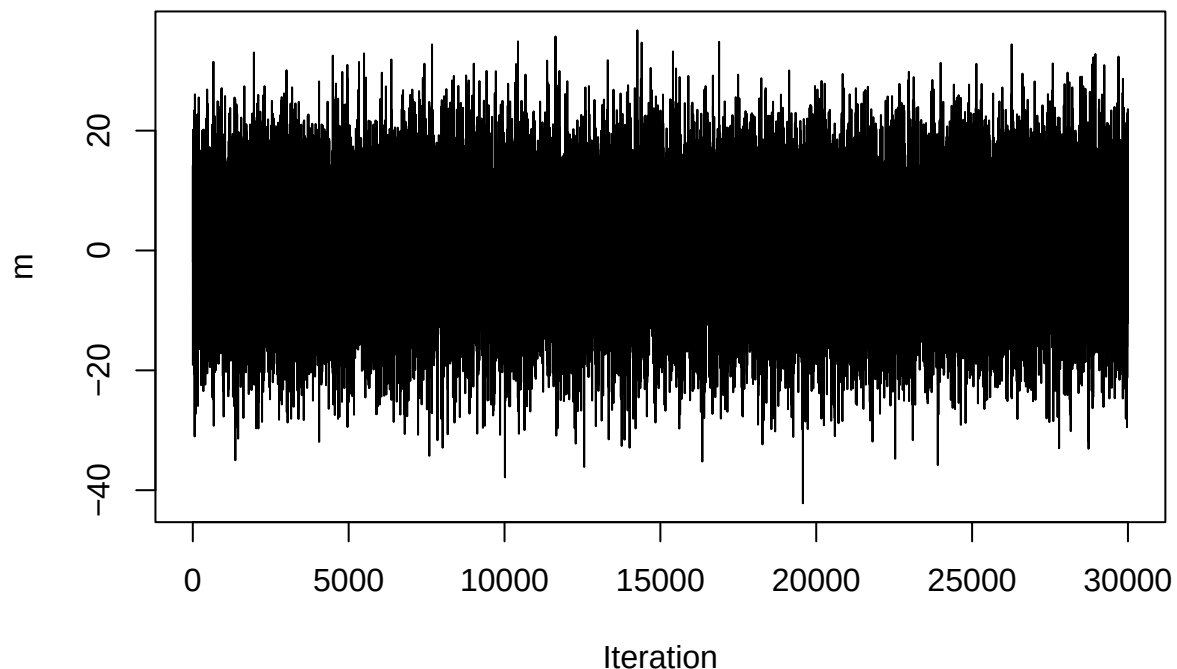
```

    theta <- theta.update(n,Y,Q,N, m)
    m.chain[i] <- m
    theta.chain[i,] <- theta
  }
  # return chains
  out <- list(m.chain = m.chain,
             theta.chain = theta.chain)
  return(out)}

n<-10
Q<-c(0.845, 0.847, 0.880, 0.674, 0.909, 0.898, 0.770, 0.801, 0.802, 0.875)
Y<-c(64, 72, 55, 27, 75, 24, 28, 66, 40, 13)
X<-c(75, 95, 63, 39, 83, 26, 41, 82, 54, 16)
N<-Y+X

MCMC.out <- MCMC(n=n,Y = Y,
                Q=Q,
                N=N,
                m.init = 0,
                theta.init = 1-var(Y)/mean(Y),
                iters = 30000)
plot(MCMC.out$m.chain, xlab = "Iteration", ylab = "m", type = "l")

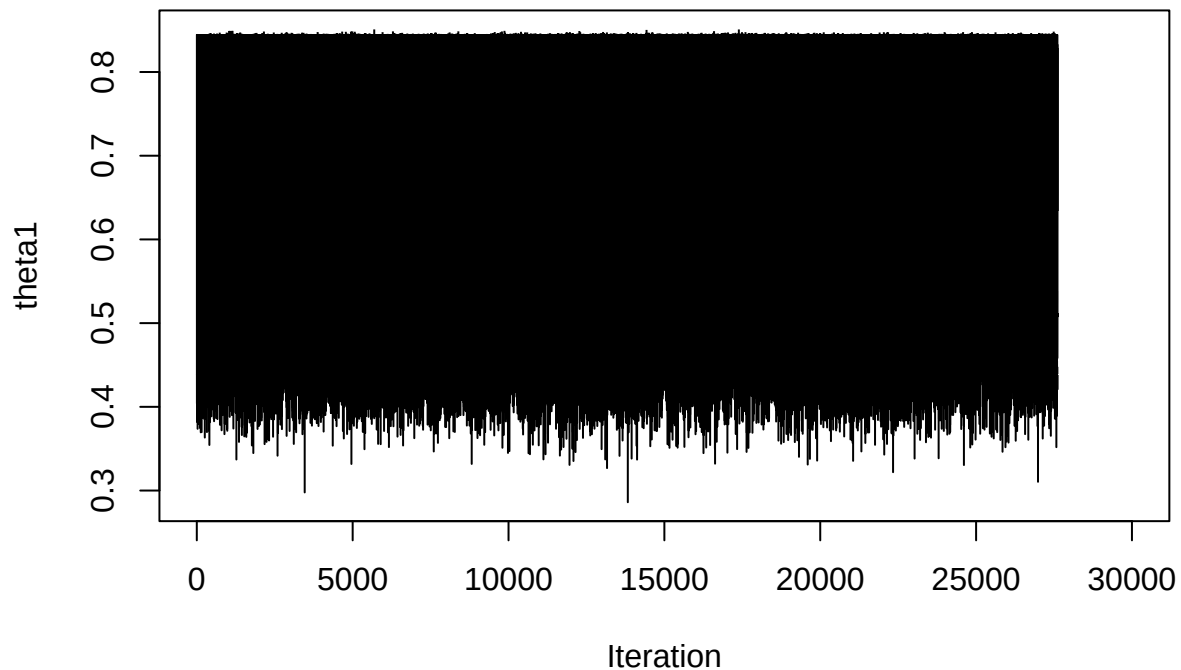
```



```

plot(MCMC.out$theta.chain[,1], xlab = "Iteration", ylab = "theta1", type = "l")

```



```
credible_interval<-matrix(NA,2,11)

for(i in 1:10)
{
  credible_interval[,i] <- quantile(MCMC.out$theta.chain[,i], c(0.025, 0.975))
}
credible_interval[,11]<-quantile(MCMC.out$m.chain, c(0.025, 0.975))
print(credible_interval)
```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]
## [1,] 0.3902155 0.3672570 0.3893348 0.3105025 0.4085152 0.3635050 0.3082013
## [2,] 0.8452042 0.8471686 0.8801992 0.6760664 0.9090738 0.8985512 0.7708121
##           [,8]      [,9]      [,10]     [,11]
## [1,] 0.3771027 0.3414115 0.3000297 -19.35266
## [2,] 0.8012669 0.8024877 0.8763788  19.63112
```

E

```
library(rjags)
n<-10
Q<-c(0.845, 0.847, 0.880, 0.674, 0.909, 0.898, 0.770, 0.801, 0.802, 0.875)
Y<-c(64, 72, 55, 27, 75, 24, 28, 66, 40, 13)
X<-c(75, 95, 63, 39, 83, 26, 41, 82, 54, 16)
N<-Y+X

data <- list(Y = Y,Q=Q,N=N, n = n)
```

```

model_string <- textConnection("model{

  # Likelihood
  for(i in 1:n){
    Y[i] ~ dbin(theta[i],N[i])
  }

  # Priors
  for(i in 1:n){
    theta[i] ~ dbeta(exp(m)*Q[i],exp(m)*(1-Q[i]))
  }
  m ~ dnorm(0,10)

}")

model <- jags.model(model_string, data = data,
                    n.chains = 2, quiet = TRUE)

update(model, 10000, progress.bar = "none")

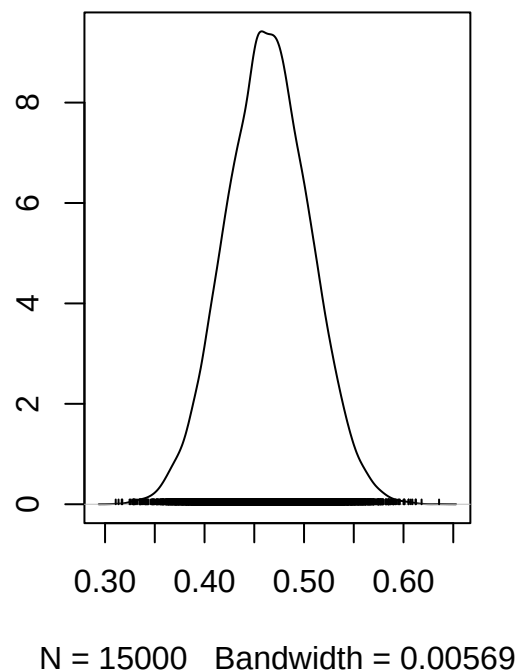
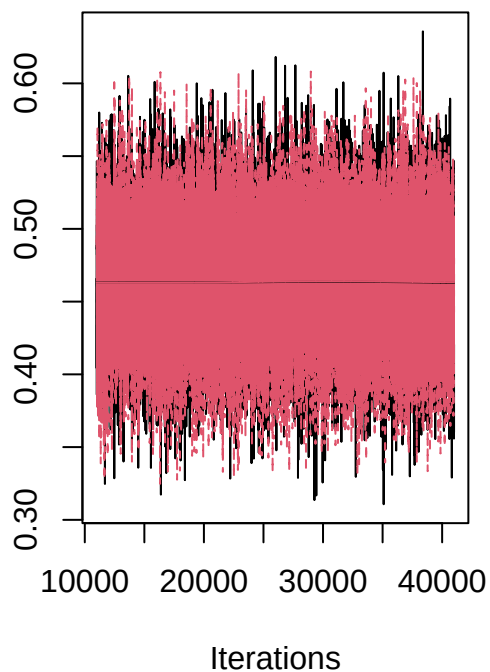
samples <- coda.samples(model, variable.names = "theta",
                        n.iter = 30000, thin = 2, progress.bar = "none")
samples2 <- coda.samples(model, variable.names = "m",
                         n.iter = 30000, thin = 2, progress.bar = "none")

summary(samples)[2]

## $quantiles
##           2.5%      25%      50%      75%      97.5%
## theta[1] 0.3813935 0.4347959 0.4634316 0.4921195 0.5464190
## theta[2] 0.3599134 0.4087152 0.4344969 0.4611200 0.5105776
## theta[3] 0.3814286 0.4398263 0.4703790 0.5013456 0.5596014
## theta[4] 0.3000239 0.3741689 0.4141530 0.4542923 0.5319188
## theta[5] 0.4019057 0.4518139 0.4785924 0.5047267 0.5561629
## theta[6] 0.3559938 0.4438305 0.4914239 0.5387595 0.6263404
## theta[7] 0.3017698 0.3714927 0.4113478 0.4514650 0.5267630
## theta[8] 0.3706315 0.4213204 0.4489695 0.4768861 0.5281455
## theta[9] 0.3331856 0.3959189 0.4298876 0.4649986 0.5314244
## theta[10] 0.2953799 0.4061594 0.4685947 0.5293768 0.6428601

plot(samples[,1])

```



```
summary(samples)[2]
```

```
## $quantiles
##           2.5%    25%    50%    75%    97.5%
## theta[1] 0.3813935 0.4347959 0.4634316 0.4921195 0.5464190
## theta[2] 0.3599134 0.4087152 0.4344969 0.4611200 0.5105776
## theta[3] 0.3814286 0.4398263 0.4703790 0.5013456 0.5596014
## theta[4] 0.3000239 0.3741689 0.4141530 0.4542923 0.5319188
## theta[5] 0.4019057 0.4518139 0.4785924 0.5047267 0.5561629
## theta[6] 0.3559938 0.4438305 0.4914239 0.5387595 0.6263404
## theta[7] 0.3017698 0.3714927 0.4113478 0.4514650 0.5267630
## theta[8] 0.3706315 0.4213204 0.4489695 0.4768861 0.5281455
## theta[9] 0.3331856 0.3959189 0.4298876 0.4649986 0.5314244
## theta[10] 0.2953799 0.4061594 0.4685947 0.5293768 0.6428601
```

We can take the 2.5% quantile as the lower boundary and 97.5% quantile as the upper boundary of the credible intervals.